Project 4: Advanced Tracking

Katarina Brešar

I. Introduction

In this assignment, we addressed more complex tracking approaches by implementing a motion prediction model using a Kalman filter, and a sequential optimization tracker based on a Particle filter. First, we experimented with Kalman filters outside of the tracking scenario to analyze the properties of Random Walk (RW), Nearly-Constant Velocity (NCV), and Nearly-Constant Acceleration (NCA) motion models on synthetically generated curves. In the second part, we implemented a Particle filter tracker that utilizes the NCV motion model and a color histogram visual model. We integrated our tracker into the VOT-lite evaluation toolkit and analyzed its performance across different parameters, motion models, and sample sizes on the VOT2014 dataset.

II. Experiments

A. Motion Models and Kalman Filters

We began by testing the Kalman filter on an artificial spiral curve using three different motion models: RW, NCV, and NCA. The matrices governing these systems (X, F, Φ, L, Q, H) are detailed in the Appendix.The behavior of the Kalman filter was heavily dependent on the process noise covariance (q) and the measurement noise covariance (r). Setting a high q (e.g., $q = 100.0, r = 1.0$) caused the filter to heavily trust the noisy measurements, resulting in a jagged, unfiltered trajectory regardless of the motion model. Conversely, lowering q and increasing r (e.g., $q = 1.0, r = 5.0$) smoothed the trajectory significantly, as the filter trusted its internal motion predictions over the noisy measurements.Comparing the models, RW performed poorly on sharp turns, often lagging behind the measurements because it assumes velocity is purely noise-driven. NCV successfully tracked the spiral by maintaining momentum, yielding the best balance of smoothing and accuracy. NCA overcompensated during tight curves because it modeled acceleration, causing it to overshoot when the curve's radius changed rapidly. To further evaluate the models, we tested the Kalman filter on a step/rectangle path and a sine wave. On the rectangle path, NCV struggled slightly at the 90-degree corners, heavily overshooting due to its momentum, whereas RW quickly adjusted to the new stationary coordinates. On the sine wave, NCV performed smoothly, while NCA provided the tightest tracking of the continuous directional shifts, though it was highly sensitive to noise spikes.

B. Particle Filter Tracker

The Particle filter was implemented by maintaining N particles representing possible states of the target. At each frame, we resampled the particles based on their weights, propagated them using the selected motion model with added Gaussian noise (sampled from Q), and recalculated their weights using the Hellinger distance between the extracted color histogram of the particle patch and the target's visual model. The final bounding box was determined by a weighted average of the particles.We used an Epanechnikov kernel for histogram extraction to give more importance to the center of the patch, and updated the target visual model at each frame using an exponential moving average ($\alpha = 0.05$).

1) Baseline Performance: Using the NCV motion model, $N = 100$ particles, and an HSV color space, we tested the tracker on 5 sequences from the VOT dataset. The parameter q was set proportionally to the target size ($q = 5.0$).

The tracker proved robust on sequences with consistent target color and smooth motion (e.g., car, jogging). However, in sequences with rapid scale changes or severe illumination shifts (e.g., hand), the color histogram visual model struggled, resulting in lower overlap and more failures. The average speed was approximately 69 FPS, which is computationally heavier than the correlation filter from Exercise 3 due to the multiple patch extractions required per frame.

C. Analysis of Tracker Performance

Motion Models within the Tracker: We compared the tracker's accuracy and robustness by swapping the underlying motion models (Table II).

NCV significantly outperformed RW and NCA. RW failed when targets moved rapidly, as the particles only spread via noise and did not carry momentum, causing the target to leave the particle spread. NCA performed worse than NCV because the acceleration state variables became highly unstable given the noise in bounding box updates, scattering the particles too widely.The parameter q dictates the spread of the particles. We found it must be proportional to the target size; if q is too small, particles collapse into a tight cluster and cannot recover if the target makes sudden movements. If q is too large, particles spread across the entire image, capturing background noise and causing catastrophic drifting.Number of Particles (N): We observed how the number of samples impacted performance.

Using too few particles ($N = 20$) resulted in poor state representation, causing frequent failures. Increasing N to 100 drastically improved accuracy and robustness. However, increasing N beyond 100 yielded diminishing returns in overlap while causing tracking speed to plummet linearly, as histogram extraction is required for every particle.

III. Conclusion

In this exercise, we implemented advanced tracking utilizing Kalman and Particle filters. We found that the Nearly-Constant Velocity (NCV) model provides the best motion prior for standard visual objects, balancing momentum and stability. Furthermore, optimizing the number of particles is crucial; $N = 100$ served as the ideal "sweet spot" providing robust tracking at near real-time speeds (69 FPS). While the color histogram is a reliable visual model, it remains susceptible to background distractors of similar colors, suggesting that combining it with a spatial template could further improve robustness.